

Project Requirement Document (PRD)

Global Meeting Time Converter Web App

(Frontend-First Architecture with Supabase and Static Hosting)

1. Project Overview

Purpose:

Develop a frontend-centric web app that converts and visualizes scheduled meeting times across time zones with an interactive world map. Backend functionality such as data persistence and sharing URL encoding will be handled via Supabase (free tier). The app will be hosted on Vercel or GitHub Pages for free.

Business Justification:

Using a frontend-first approach with serverless backend services minimizes infrastructure costs and development overhead, enabling rapid deployment and scalability.

2. Goals and Objectives

- Deliver a fully functional, performant frontend app with all core features.
- Use Supabase for lightweight backend needs (sharing links, optional user data).
- Host on free static hosting platforms (Vercel or GitHub Pages).
- Maintain responsive, accessible, and intuitive UI/UX.
- Ensure easy sharing and exporting features without a traditional backend server.

3. Functional Requirements

(unchanged from previous PRD regarding user input, time conversion, map visualization, sharing, UI)

4. Technical Requirements (Updated)

4.1 Frontend

- Framework: React (recommended for ecosystem support), Vue or Angular are alternatives.
- Timezone handling: `date-fns-tz` or `luxon` (modern, tree-shakeable).

- Map Integration: Leaflet.js (open source, no API key needed) or Mapbox (free tier with API key).
- State management: React Context or lightweight state libraries (if needed).
- UI Framework: Tailwind CSS or similar for responsive design.

4.2 Backend (Serverless)

- **Supabase (Free Tier):**
 - Use Supabase Realtime Database or Postgres for storing sharing links and meeting configurations.
 - Use Supabase Auth only if user accounts are needed (optional, not required initially).
 - Serverless functions (Supabase Edge Functions) can be used for URL encoding/decoding logic if needed.
- **Sharing Links:**
 - Store meeting time and participants data as JSON in Supabase with a unique short ID.
 - Frontend fetches data by ID to reconstruct meeting schedules.

4.3 Hosting

- **Vercel:**
 - Recommended for React apps with serverless function support (free tier).
 - Supports automatic deployments from GitHub.
 - Enables edge functions for lightweight backend logic if needed.
- **GitHub Pages:**
 - Suitable for purely static frontend apps without serverless functions.
 - If backend functions are needed, must rely fully on Supabase or external APIs.

5. Architecture Overview

```

text
User Browser
  |
Frontend React App (hosted on Vercel or GitHub Pages)

```

| (reads/writes)
Supabase (Database + Edge Functions)
|
Timezone Libraries + Map Library (Leaflet.js)

- All UI and logic run in the browser.
- Supabase handles persistent storage of meeting data for sharing.
- No dedicated backend server needed.

6. User Experience and Design

(unchanged, with emphasis on fast load times and offline resilience where possible)

7. Constraints and Assumptions

- Supabase free tier limits:
 - 500MB database storage, 2GB bandwidth/month, 50k monthly active users (sufficient for MVP).
 - Rate limits on API calls and edge functions apply; app should handle gracefully.
- Map library choice affects API key requirements: Leaflet is fully open-source with no key; Mapbox requires free API key.
- GitHub Pages cannot run serverless functions; if sharing requires backend logic, Vercel is preferred.

8. Milestones and Timeline (Updated)

Milestone	Duration	Notes
Requirements & UI/UX Design	2 weeks	Including map and sharing flow design
Frontend Development	3 weeks	Implement time conversion, map, sharing UI
Supabase Integration	1 week	Setup DB schema, implement sharing storage
Testing & Bug Fixing	1 week	Performance, accessibility, API limits
Deployment & Launch	1 week	Vercel or GitHub Pages setup and release

9. Risks and Mitigation

Risk	Mitigation Strategy
Supabase free tier limits exceeded	Monitor usage, optimize data size, consider paid plan if needed
Map API key limits or costs (if Mapbox used)	Prefer Leaflet for no-key solution or monitor usage carefully

Lack of backend flexibility on GitHub Pages

Use Vercel for serverless functions if needed

Client-side timezone calculations complexity

Use well-tested timezone libraries, cache data as needed

10. Stakeholders and Roles

(unchanged)

11. Appendices

- Supabase schema example for sharing links table:

sql

```
CREATE TABLE meetings (
```

- `id` uuid **PRIMARY KEY DEFAULT** uuid_generate_v4(),
- `slug` **text UNIQUE** NOT NULL,
- `client_timezone` **text** NOT NULL,
- `meeting_datetime` **timestamp**tz NOT NULL,
- `participants` **jsonb** NOT NULL,
- `created_at` **timestamp**tz **DEFAULT** now()
- `);`
-

- Sample frontend flow for saving and loading meeting data via Supabase.
- Leaflet.js integration examples.